

Boolean Arithmetic



Building a Modern Computer From First Principles

1

Counting systems

quantity	decimal	binary	3-bit register
	0	0	000
*	1	1	001
**	2	10	010
***	3	11	011
****	4	100	100
*****	5	101	101
*****	6	110	110
*****	7	111	111
*****	8	1000	overflow
*****	9	1001	overflow
*****	10	1010	overflow

3

Rationale

$$(9038)_{ten} = 9 \cdot 10^3 + 0 \cdot 10^2 + 3 \cdot 10^1 + 8 \cdot 10^0 = 9038$$

$$(10011)_{two} = 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 19$$

$$(x_n x_{n-1} \dots x_0)_b = \sum_{i=0}^n x_i \cdot b^i$$

4

Binary addition

Assuming a 4-bit system:

$$\begin{array}{r} 0 \ 0 \ 0 \ 1 \\ \hline 1 \ 0 \ 0 \ 1 \\ 0 \ 1 \ 0 \ 1 \\ \hline \end{array} +$$

0 1 1 1 0

no overflow

$$\begin{array}{r} 1 \ 1 \ 1 \ 1 \\ \hline 1 \ 0 \ 1 \ 1 \\ 0 \ 1 \ 1 \ 1 \\ \hline \end{array} +$$

1 0 0 1 0

overflow

- Algorithm: exactly the same as in decimal addition
- Overflow (MSB carry) has to be dealt with.

5

Representing negative numbers (4-bit system)

0	0000		
1	0001	1111	-1
2	0010	1110	-2
3	0011	1101	-3
4	0100	1100	-4
5	0101	1011	-5
6	0110	1010	-6
7	0111	1001	-7
		1000	-8

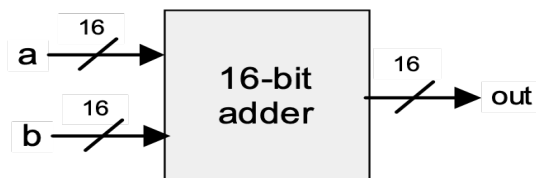
- The codes of all positive numbers begin with a "0"
- The codes of all negative numbers begin with a "1"
- To convert a number:
leave all trailing 0's and first 1 intact,
and flip all the remaining bits

Example: $2 - 5 = 2 + (-5) =$

$$\begin{array}{r} 0010 \\ +1011 \\ \hline 1101 = -3 \end{array}$$

6

Building an Adder chip

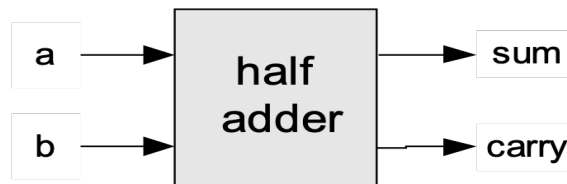


- Adder: a chip designed to add two integers
- Proposed implementation:
 - Half adder: designed to add 2 bits
 - Full adder: designed to add 3 bits
 - Adder: designed to add two n-bit numbers.

7

Half adder (designed to add 2 bits)

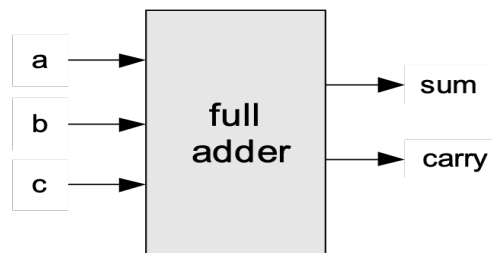
a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Implementation: based on two gates that you've seen before.

Full adder (designed to add 3 bits)

a	b	c	sum	carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Implementation: can be based on half-adder gates.

n-bit Adder (designed to add two 16-bit numbers)

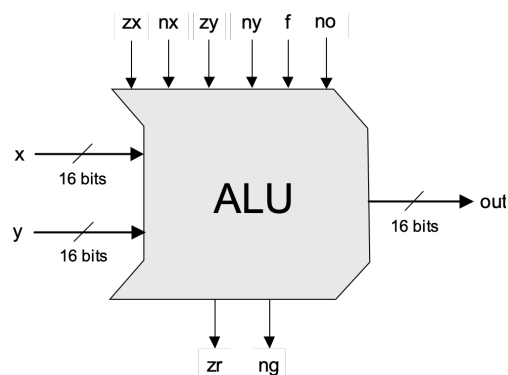
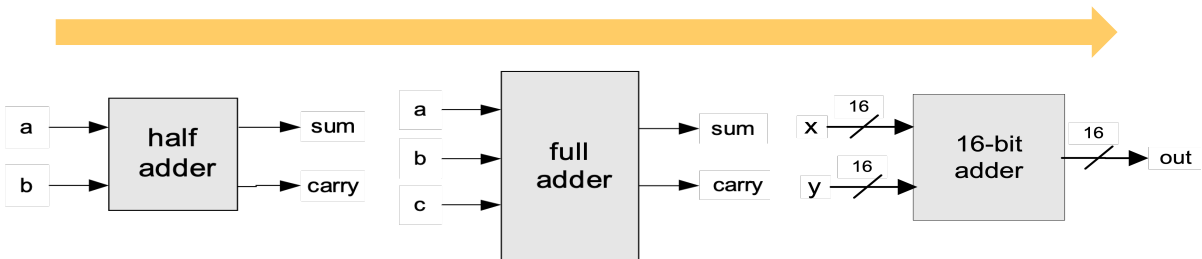


...	1	0	1	1	a
					+
...	0	0	1	0	b
...	1	1	0	1	out

Implementation: array of full-adder gates.

10

The ALU (of the Hack platform)



`out(x, y, control bits) =`

`x+y, x-y, y-x,`
`0, 1, -1,`
`x, y, -x, -y,`
`!x, !y,`
`x+1, y+1, x-1, y-1,`
`x&y, x|y`

11

ALU logic (Hack platform)

These bits instruct how to pre-set the x input		These bits instruct how to pre-set the y input		This bit selects between + / And	This bit inst. how to post-set out	Resulting ALU output
zx	nx	zy	ny	f	no	out=
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x And y	if no then out=!out	f (X, y) =
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1						y
0						!x
1						!y
0						-x
1						-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x y

Implementation: build a logic gate architecture that "executes" the control bit "instructions": if zx==1 then set x to 0 (bit-wise), etc.

12

Project 2: combinational chips

Chip (HDL)	Description	Test script	Compare file
HalfAdder	Half Adder	HalfAdder.tst	HalfAdder.cmp
FullAdder	Full Adder	FullAdder.tst	FullAdder.cmp
Add16	16-bit Adder	Add16.tst	Add16.cmp
Inc16	16-bit incrementer	Inc16.tst	Inc16.cmp
ALU	Arithmetic Logic Unit (complete)	ALU.tst	ALU.cmp

15

With a little help from my friends - understanding ALU

zx	nx	zy	ny	f	no	out
if zx then x = 0	if nx then x = !x	if zy then y = 0	if ny then y = !y	if f then out = x+y else out = x&y	if no then out = !out	f(x, y)
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y
0	1	1	1	1	1	x + 1
1	1	0	1	1	1	y + 1
0	0	1	1	1	0	x - 1
1	1	0	0	1	0	y - 1
0	0	0	0	1	0	x + y
0	1	0	0	1	1	x - y
0	0	0	1	1	1	y - x
0	0	0	0	0	0	x & y
0	1	0	1	0	1	x y

16

With a little help from my friends - understanding ALU

zx	nx	zy	ny	f	no	out
if zx then x = 0	if nx then x = !x	if zy then y = 0	if ny then y = !y	if f then out = x+y else out = x&y	if no then out = !out	f(x, y)

■ variables:

- x; y
- !x = $(-x)_{c1} = (-x - 1)_{c2}$; !y = $(-y)_{c1} = (-y - 1)_{c2}$

■ constants:

- 0 (zero)
- !(a = 0) = -1

■ operators:

- +(x, y); &(x, y)
- !(out) = $(-out)_{c1} = (-out - 1)_{c2}$

NB: zx and nx (similarly, zy and ny) act in sequence:

- !x is applied to x=x or x=0
- !y is applied to y=y or y=0

With a little help from my friends - understanding ALU

zx	nx	zy	ny	f	no	out
if zx then x = 0	if nx then x = !x	if zy then y = 0	if ny then y = !y	if f then out = x+y else out = x&y	if no then out = !out	f(x, y)
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y

With a little help from my friends - understanding ALU

zx	nx	zy	ny	f	no	out
if zx then x = 0	if nx then x = !x	if zy then y = 0	if ny then y = !y	if f then out = x+y else out = x&y	if no then out = !out	f(x, y)
0	1	1	1	1	1	x + 1
1	1	0	1	1	1	y + 1
0	0	1	1	1	0	x - 1
1	1	0	0	1	0	y - 1
0	0	0	0	1	0	x + y
0	1	0	0	1	1	x - y
0	0	0	1	1	1	y - x
0	0	0	0	0	0	x & y
0	1	0	1	0	1	x y

With a little help from my friends - designing ALU

■ The designer perspective

- 1- settle the **instruction set** for the machine
- 2- empower the **ALU** with the required operations
- 3- derive the **bit configuration** for each operation

■ Hack Computer ALU

- 2 operands, 1 output, 6 control bits, 2 status bits
inputs: x, y; **output:** f(x,y)
control bits: zx, nx, zy, ny, f, no; **status bits:** zr, ng
- Instruction set:
0, 1, -1, x, y, !x, !y, -x, -y, x+1, y+1, x-1, y-1, x+y, x-y,
y-x, x&y, xly

22

With a little help from my friends - designing ALU

■ Operations design examples:

- **-1** $\rightarrow (-1) + 0 = -1$; **0 + (-1) = -1**
zx=1, nx=1; zy=1, ny=0; f=1; no=0
zx=1, nx=0; zy=1, ny=1; f=1; no=0 (not in table)
- **1** $\rightarrow (-1) + (-1) = -2 \rightarrow !(-2) = (2-1) = 1$
zx=1, nx=1; zy=1, ny=1; f=1; no=1
- **x-1** $\rightarrow x + (-1) = x - 1$
zx=0, nx=0; zy=1, ny=1; f=1; no=0
- **x+1** $\rightarrow !(x)+(-1)=(-x - 1)-1=-x-2 \rightarrow !(-x-2)=x+2-1=x+1$
zx=0, nx=1; zy=1, ny=1; f=1; no=1

23

With a little help from my friends - designing ALU

■ Operations design examples:

- $x-y \rightarrow !(x)+y=(-x-1)+y=-x+y-1 \rightarrow$
 $!(-x+y-1)=x-y+1-1=x-y$
 $zx=0, nx=1; zy=0, ny=0; f=1; no=1$
- $y-x \rightarrow x+!(y)=x+(-y-1)=x-y-1 \rightarrow$
 $!(x-y-1)=-x+y+1-1=y-x$
 $zx=0, nx=0; zy=0, ny=1; f=1; no=1$
- $x \mid y \rightarrow !(x) \& !(y) = !(x \mid y) \rightarrow !(x \mid y) = x \mid y$
 $zx=0, nx=1; zy=0, ny=1; f=0; no=1$